

SaaS Context Broker

Full project context dossier

This document is a complete working context for the repository: what it does, how the code is structured, the runtime flows, current capabilities, known weaknesses, and the most important improvements to resume development safely.

Generated: 2026-06-11 10:57

Repository: D:\devProject\contextBroker

1. Executive summary

The project is a backend incident context broker built with Spring Boot 4.0.2 and Java 21. It ingests Slack messages and Jira status updates, keeps incidents in PostgreSQL, scores them using configurable rules, generates an audit timeline, and sends Slack notifications when incidents become high risk. It also exposes dashboard-style analytics endpoints for incident status, severity, risk, trends, and system health.

In practical terms, this is already more than a toy demo: it has a layered service architecture, persistence, validation, exception handling, scheduled background processing, and a reasonably rich API surface. The main missing pieces are production hardening, testing, auth, and removal of a few implementation inconsistencies.

2. What the product is meant to do

Its purpose is to act as the middle layer between operational SaaS tools and an incident dashboard. Slack and Jira updates are treated as signals about the same incident. The broker centralizes those signals, computes risk, keeps track of lifecycle state, and pushes notifications when risk crosses a threshold.

This makes the project a domain-focused incident operations backend rather than a generic CRUD app. The business value is in correlating updates, preserving timeline context, and giving operators a single place to inspect health and urgency.

3. Implemented capability map

Capability	What it does now
Slack ingestion	POST /incoming/slack accepts incidentKey + message, creates incidents if needed, evaluates severity, and
Jira ingestion	POST /incoming/jira accepts incidentKey + JiraStatus, updates lifecycle state, and logs a Jira event.
Webhook config	POST /config/slack stores a Slack webhook URL after validation.
Incident lookup	GET /incident/{incidentKey} returns the current incident summary.
Detailed incident view	GET /incident/incident-details/{incidentKey} returns a richer read model.
Timeline	GET /incident/{incidentKey}/timeline returns merged timeline records from new and legacy sources.
Metrics	GET /incident/metrics returns totals, active, critical, stale, resolved, and escalation count (currently stubbed)
Health	GET /incident/system-health classifies the overall system as HEALTHY/DEGRADED/CRITICAL.
Priority / risk	GET /incident/risk-score/{incidentKey} and /priority/{incidentKey} expose computed operational importance
Highest risk	GET /incident/highest-risk returns the currently riskiest active incident.
Distribution and trend	GET /incident/severity-distribution and /trend provide analytics for dashboards.
Assignment	POST /incident/{incidentKey}/assign assigns or reassigns ownership.
Automation	IncidentMonitor runs on a schedule and escalates stale critical incidents via Slack.

4. How the code is organized

Package	Role
controller	HTTP layer: SlackController, JiraaController, IncidentController, WebhookConfigController
service	Business logic: IncidentService, TimelineService, SlackNotifier, WebhookConfigService, IncidentMonitor
repository	JPA repositories for incidents, timelines, and webhook config
entity	Database-backed domain objects
dto	API request/response shapes

Package	Role
model	Enums for severity and lifecycle status
exception	Global error handling and not-found exception
config	CORS and web MVC configuration
specification	Dynamic incident filtering with JPA criteria

5. Runtime flow

Incoming Slack path:

Slack webhook -> SlackController -> IncidentService.handleUpdate -> timeline event -> severity evaluation -> optional Slack alert -> persistence.

Incoming Jira path:

Jira webhook -> JiraaController -> IncidentService.handleJiraUpdate -> lifecycle update -> timeline event -> severity evaluation -> optional alert -> persistence.

Scheduled monitoring path:

IncidentMonitor scans all incidents every 10 seconds, skips resolved/closed cases, re-evaluates stale incidents, and sends escalation Slack alerts when cooldown allows.

6. Domain model and data

Entity / enum	Meaning
IncidentEntity	Core incident aggregate with incidentKey, message, timestamps, Jira status, incident status, severity, assignee
TimelineEventEntity	Current unified event history format with event type, source, description, content, timestamp.
IncidentEventEntity	Legacy event table still read by timeline API for backward compatibility.
WebhookConfig	Stores the Slack webhook URL.
Severity	LOW, MEDIUM, HIGH, CRITICAL.
IncidentStatus	DETECTED, CREATED, IN_PROGRESS, RESOLVED, CLOSED.
JiraStatus	OPEN, IN_PROGRESS, RESOLVED, CLOSED.

The model is simple and workable, but there is some duplication: incident status and severity overlap conceptually, and legacy/new timeline sources coexist. That is fine for now, but it is a clear cleanup target once the historical migration is complete.

7. Scoring and alerting logic

The severity engine is the most important business logic in the project. It uses configurable weights from application.properties: urgent=50, jiraOpen=30, jiraInProgress=10, stale=20. The score is mapped to severity thresholds: >=70 CRITICAL, >=50 HIGH, >=30 MEDIUM, otherwise LOW.

The project also exposes a separate risk score calculation, which adds different weight adjustments and caps the result at 100. That means the read-time risk score and the alert-time severity score are not identical, which is an important behavior to remember when debugging or extending the system.

Alerts are sent only for HIGH and CRITICAL incidents, and only when the cooldown window permits it. The notifier currently posts to a Slack incoming webhook URL stored in the database.

8. API surface at a glance

Area	Endpoints
Inbound webhooks	POST /incoming/slack, POST /incoming/jira
Configuration	POST /config/slack
Incident reads	GET /incident/{incidentKey}, GET /incident/incident-details/{incidentKey}, GET /incident/{incidentKey}
Analytics	GET /incident/metrics, /system-health, /highest-risk, /severity-distribution, /trend
Scoring	GET /incident/risk-score/{incidentKey}, GET /incident/priority/{incidentKey}
Operations	GET /incident/all, POST /incident/{incidentKey}/assign

9. Configuration and environment

The app is configured for PostgreSQL on localhost:5432/context_broker with username/password postgres/postgres. Hibernate DDL auto-update is enabled. CORS allows http://localhost:3000. The scheduling system is enabled at application startup.

The scoring weights and staleness threshold are already externalized, which is good. But database credentials, allowed origins, alert cooldowns, and monitor cadence still need better configuration handling for real deployments.

10. What works well

- Clear responsibility split between controllers, services, repositories, and entities.
- Useful operational API surface instead of just a single ingestion endpoint.
- Configurable scoring inputs.
- Timeline/audit trail support.
- Automatic escalation for stale critical incidents.
- Validation and global exception responses.
- Paginated incident listing with filters.

11. What is weak or risky

- No authentication or authorization.
- No webhook signature verification.
- RestTemplate notification path has no retry or backoff.
- Monitoring and scoring logic are not fully consistent with each other.
- A few values are hardcoded instead of config-driven.
- Testing is almost absent.
- The project still carries legacy artifacts and some naming inconsistencies.
- escalationsToday is not implemented.
- System.out.println is used where structured logging should exist.

12. Features that look missing

- Auth/RBAC for operational endpoints.
- Source validation for Slack/Jira webhooks.
- Retries, timeout control, and failure tracking for Slack notifications.
- User/team entities instead of free-text assignees.
- OpenAPI/Swagger docs.
- Integration tests and unit tests for the engine.
- Better production config management and secrets handling.
- More advanced search/sort/reporting.
- A proper migration path for legacy timeline data.

13. Recommended next work

Priority	Recommendation
P1	Unify scoring so read-time, alert-time, and monitor-time logic all share one source of truth.
P1	Add auth, authorization, and webhook verification before exposing this beyond local development.
P1	Move hardcoded operational values into configuration.
P2	Build real tests around scoring, handlers, and controller validation.
P2	Replace print statements with structured logging and add retries for outbound notifications.
P2	Add API documentation and cleaner developer onboarding.
P3	Remove or retire legacy event storage once migration is complete.

14. File references to resume quickly

- pom.xml
- src/main/resources/application.properties
- src/main/java/.../SaaSContextBrokerApplication.java
- src/main/java/.../controller/*.java
- src/main/java/.../service/IncidentService.java
- src/main/java/.../service/IncidentMonitor.java
- src/main/java/.../service/TimelineService.java
- src/main/java/.../entity/*.java
- src/main/java/.../dto/*.java
- src/main/java/.../exception/GlobalExceptionHandler.java
- src/main/java/.../specification/IncidentSpecification.java

15. Bottom line

This project is a coherent incident intelligence backend with real value: it ingests, correlates, classifies, tracks, and escalates incidents. If you resume development, the biggest wins will come from hardening security, eliminating duplicated logic, turning the stubbed metrics into real metrics, and covering the core flows with tests.